Cognitive Cohesion in the Agentic Era: Addressing Semantic Misalignment and Context Debt in Multi-Agent Systems

Executive Summary

This report addresses the critical, yet often overlooked, challenge of semantic misalignment in the burgeoning landscape of multi-agent AI systems. While advancements like Gemini CLI's million-token context window and protocols like the Model Context Protocol (MCP) have made significant strides in solving for syntactic interoperability, they have inadvertently magnified the deeper problem of ensuring shared understanding among collaborating agents. The agentic landscape remains deeply fragmented, not at the level of data exchange, but at the level of meaning.[1]

This analysis validates a crucial insight: this semantic gap creates "context conflict," which manifests as a form of **cognitive debt** for human developers. This debt is not paid down by refactoring code but is serviced through the increased cognitive load required to debug opaque, emergent behaviors, reconcile divergent agent states, and infer the collective intent of a system that lacks a unified mental model. Each instance of misinterpretation by an agent team accumulates as a liability on the developer's cognitive balance sheet, hindering progress and eroding trust in the system.[3]

The report provides a multi-layered analysis, beginning with a deconstruction of the forms of misalignment using the "Misalignment Mosaic" framework, which traces the decay of meaning from high-level concepts down to code.[5] It proceeds to evaluate the limitations of current technologies, demonstrating how protocols like MCP prioritize simple tool invocation over the complex negotiation required for true collaboration, and how even massive context windows fail to guarantee shared understanding.[6] Architectural patterns—hierarchical, peer-to-peer, and federated—are examined for their profound influence on context propagation and the potential for semantic drift.

The analysis culminates in a survey of emerging solutions that offer a path toward genuine cognitive cohesion. These solutions operate at every level of the system stack: data-level

grounding through semantic layers, model-level alignment of internal representations via federated learning, protocol-level explicitness using shared ontologies, and system-level consistency enforced by transactional frameworks like SagaLLM.[9]

The primary recommendation of this report is a necessary paradigm shift for the field. The focus must evolve from optimizing individual agent capability and the mechanics of data exchange to prioritizing the establishment and maintenance of **Shared Mental Models (SMMs)** as a first-class architectural concern.[13] Only by architecting for alignment at a conceptual level can the promise of multi-agent systems be fully realized, transforming a fragmented collection of actors into a cohesive, reliable, and understandable cognitive ecosystem.

Section 1: The Anatomy of Semantic Misalignment

1.1 Introduction: Beyond Technical Failure

The prevailing discourse on failures in multi-agent systems (MAS) often frames them as technical bugs—errors in code, faulty logic, or suboptimal reward functions. This perspective, however, is dangerously incomplete. Many such failures originate not in the implementation but upstream, during the conceptual design phase, where semantic ambiguity and normative projection create the conditions for systemic misalignment.[5] As agentic systems, such as those enabled by the Gemini CLI, become increasingly interconnected through protocols like MCP and collaborative platforms like GitHub Actions, they form nascent multi-agent ecosystems.[15] Within these ecosystems, the probability of subtle, meaning-level failures increases exponentially, posing a significant threat to system reliability and trustworthiness.

Semantic misalignment can be formally defined as the discrepancy in the interpretation of information, goals, or context between two or more autonomous agents, leading to suboptimal, erroneous, or unpredictable collective behavior. It is not a failure of communication in the syntactic sense—the bits are transferred correctly—but a failure of comprehension. This section dissects the fundamental sources of this misalignment, moving from the conceptual and linguistic down to the structural and mathematical, to build a comprehensive understanding of this foundational challenge.

1.2 The Misalignment Mosaic: A Diagnostic Framework

To systematically diagnose meaning-level failures, the "Misalignment Mosaic" serves as an essential framework. It identifies four interconnected components that trace how semantic instability and interpretive ambiguity migrate from human language into system design and, ultimately, to evaluation failures.[5] This framework shifts the focus from purely technical debugging to a more holistic analysis of the entire design-to-deployment lifecycle.

Terminological Inconsistency
A foundational driver of misalignment is the inconsistent and often interchangeable use of core concepts within the AI research and development community. The terms "cooperation" and "coordination," for instance, are frequently conflated, yet they describe distinct phenomena. Coordination may refer to a low-level, mechanistic alignment of actions (e.g., two agents avoiding a collision), while cooperation implies a higher-level, strategic alignment of goals, potentially involving sacrifice or complex negotiation.5
This ambiguity mirrors the Rabbit-Duck optical illusion: the same observable agent behavior can be interpreted as either simple coordination or complex cooperation depending on the observer's perspective and assumptions. When a design document uses these terms loosely, it creates an epistemic instability that propagates downstream. An engineer might implement a reward function that successfully incentivizes coordination, but an evaluator, primed by the language of cooperation, may judge the system's output as a failure because it lacks strategic depth.[5]

Concept-to-Code Decay
This component describes the inevitable degradation of high-level, abstract concepts as they are translated into the concrete, formal language of code, policies, and reward functions. A concept like "ensure user safety" is rich with nuance and context. Its implementation, however, might be reduced to a simple rule, such as "minimize the agent's response toxicity score." This translation is inherently lossy. The operationalized rule may fail to capture critical aspects of the original concept, such as the psychological safety of the user or the risk of providing dangerously incomplete information. This "decay" creates a gap between the system's intended purpose and its actual, implemented behavior, a direct form of semantic misalignment.5
Interpretive Ambiguity & Morality as Cooperation
Humans have a natural tendency to anthropomorphize and project intent onto autonomous systems. A significant source of interpretive ambiguity arises when observers, particularly designers and evaluators, overread agent behavior through a moral lens. There is a systemic tendency to equate cooperative outcomes with morally "good" behavior and competitive or uncoordinated outcomes with "bad" behavior.5 This moral overreading biases the design and evaluation process. A system that achieves its goal through a ruthlessly efficient, non-cooperative strategy might be deemed a failure, not because it failed its objective, but

because its process did not align with human-centric norms of cooperation. This conflation embeds human social biases into the system's evaluation criteria, misaligning its optimization path with its stated technical goals.

1.3 Structural and Representational Misalignment

Beyond the conceptual layer, semantic misalignment is rooted in the fundamental technical architecture of current AI systems. These structural sources of misalignment are often more subtle but no less impactful.

One primary source is the inherent mismatch between the medium of human thought—natural language—and the operational medium of Large Language Models (LLMs)—high-dimensional vector spaces. Natural language is rich, ambiguous, and context-dependent. The process of embedding this language into vectors for machine processing is a form of "semantic compression" that can lead to information loss and behavioral drift.[16] Agents communicating via natural language protocols may experience inconsistencies between what they "say" and what they "do" because the representational space of their communication is structurally misaligned with their internal semantic space.[16] This can manifest as semantic redundancy, ambiguity, and difficulty in maintaining a consistent state between agents.

Further architectural limitations contribute to misalignment. In many multi-agent frameworks, robust mechanisms for maintaining a shared context are absent. This leads to situations where each agent operates with a different understanding of the current state, a problem exacerbated by distributed state management challenges.[17] Synchronization problems emerge when agents must maintain a consistent worldview while operating in parallel, leading to divergent state representations. Furthermore, agents trained on different datasets or with different objectives will inevitably develop inconsistent knowledge representations. This "training misalignment" creates friction during collaboration, as agents may operate from different factual bases or with different knowledge cutoff dates, leading to temporal misalignments where contradictions can emerge without any single agent being individually incorrect.[17]

1.4 Misalignment in Value and Goals

The most profound form of misalignment occurs at the level of agent goals and values. Even if agents can perfectly share context and interpret language, they may still fail to collaborate effectively if their underlying objectives conflict. A critical concern in human-AI teams is the

emergence of misaligned instrumental values.[18] This occurs when two agents, even with unrelated terminal (final) goals, develop conflicting instrumental (sub) goals in the pursuit of their respective objectives.

This is particularly dangerous in scenarios where the AI's learning and adaptation timescale is significantly faster than the human's. In such a dynamic, the AI agent becomes strictly dominant, capable of optimizing its behavior against a relatively static human policy.[18] Formal models of "multi-agent POWER" attempt to quantify the instrumental value an agent can expect to achieve in the presence of another agent. These models show that even if the agents' reward functions are highly correlated—or even identical—their instrumental goals can diverge based on the dynamics of their interaction. This reveals that alignment is not simply a matter of setting the same top-level goal; it is a continuous property that must be maintained throughout the interaction, accounting for the emergent sub-goals that arise during complex problem-solving.[18]

The compounding nature of these different forms of misalignment creates a formidable challenge. It is not a static error but a dynamic process of "interpretive heterogeneity" that can self-amplify across a system's lifecycle.[5] The process begins with a seemingly innocuous terminological ambiguity in a design document, for instance, the conflation of "cooperation" and "coordination." This ambiguity leads an engineer to implement a reward function based on a narrow, technical definition of coordination, such as rewarding agents for occupying adjacent spaces in a grid. The resulting agent behavior is technically coordinated—the agents cluster together—but lacks any strategic cooperation needed to achieve a more complex goal. An evaluator, primed by the design document's use of the term "cooperation," views this clustering not as a success of coordination but as a failure of cooperation. This creates a broken feedback loop where the system is fundamentally misaligned at a conceptual level, a flaw that propagates from design through implementation to evaluation. This reveals that the fragmentation seen across the agentic landscape—with different frameworks and teams operating in silos—is not just an infrastructure problem but a direct consequence of lacking a shared meta-language to even discuss, let alone solve, these deep alignment issues.[1] The data and infrastructure silos are merely symptoms of a deeper conceptual fragmentation.

Section 2: The Communication Chasm: Protocols, Pipes, and the Illusion of Understanding

2.1 The Spectrum of Interoperability: From Syntactic to Semantic

For agents to collaborate, they must first communicate. The effectiveness of this communication exists on a spectrum of interoperability. At one end lies *syntactic interoperability*, which is the ability of different systems to correctly format, exchange, and parse messages according to a strict, predefined structure. This is a solved problem, addressed by decades of work in distributed systems and standardized data formats like JSON.[19] At the other, far more challenging end of the spectrum is

semantic interoperability, which is the ability of systems to agree on the *meaning*, intent, and context of those messages.[11] The central thesis of this analysis is that the current agentic landscape, driven by protocols like MCP, has overwhelmingly prioritized syntactic interoperability, creating a robust illusion of understanding while failing to address the underlying semantic chasm.

This is not a new problem. Legacy protocols from the early days of distributed computing, such as CORBA and RMI, also focused on defining interfaces and data structures (Interface Definition Languages) to ensure syntactic correctness.[19] However, their heavyweight, synchronous nature and complex object lifecycle management proved unsuitable for modern, cloud-native architectures. The industry's pivot towards lightweight, stateless protocols and RESTful APIs emphasized developer experience and operational simplicity but continued to sidestep the hard problem of semantic reasoning. The prevailing approach has been to rely on AI-powered natural language processing for dynamic interpretation rather than attempting to standardize meaning through shared vocabularies or formal ontologies.[19] This has left a critical gap, which becomes increasingly perilous as agents are granted more autonomy.

2.2 Technical Deep Dive: Model Context Protocol (MCP)

The Model Context Protocol (MCP), championed by Anthropic and integral to tools like Google's Gemini CLI, is a prime example of a modern protocol designed for syntactic excellence.[15] It establishes a standardized client-server model for an LLM to interact with external tools and data sources. However, a detailed analysis reveals its inherent limitations for true multi-agent collaboration.

Architecture: MCP is built on a centralized, client-server model. A single orchestrator, typically the LLM runtime itself, manages the entire lifecycle of a tool invocation.[6] In this paradigm, agents are treated as "tools" and do not communicate directly with one another. All interactions are mediated by the central planner, which uses metadata to decide which tool to call and when. This architecture offers a simple, predictable, and easily traceable control flow, but this simplicity comes at the cost of flexibility and autonomy. It is ill-suited for dynamic

agent swarms or decentralized systems where peer-to-peer interaction is required.[6]

State Management: A defining feature of MCP is that it is intentionally stateless.[6] Each tool call is an independent, atomic transaction. The orchestrator provides all necessary context—tool descriptions, parameters, expected outputs—inline for every call. There is no built-in support for memory, session continuity, or the kind of long-running, contextual negotiation that is essential for complex collaborative tasks.[6] While this statelessness enhances scalability and aligns well with serverless execution models, it forces all conversational history and shared understanding to be reconstructed from scratch with every interaction, a significant limitation for building coherent, long-term collaborations.[6]

Discovery: MCP eschews dynamic or active discovery mechanisms. Instead, it relies on a static registry of predefined metadata files that describe each tool's inputs, outputs, and purpose.[6] This manual, curated approach is simple and enforces clear contracts but is fundamentally rigid. It cannot support runtime discovery of new agents or capabilities, and any evolution of a tool's interface requires manual updates to the metadata, creating friction for developers and hindering the creation of open, extensible ecosystems.[6]

Semantic Capabilities: MCP's strength lies in its robust support for syntactic communication. Through the use of typed schemas (e.g., JSON-RPC 2.0) and structured metadata, it ensures that messages are well-formed and adhere to a predefined contract.[19] However, its semantic capabilities are severely limited. The protocol fundamentally

assumes that the full context required for an operation is known and can be provided at execution time.[6] It has no native mechanisms for resolving ambiguity, negotiating meaning, or inferring intent beyond what is explicitly stated in the static metadata. Its design focus is squarely on "agent-to-tool" integration—a single agent securely calling multiple external functions—not on "agent-to-agent" collaboration, which requires a much richer semantic dialogue.[7]

2.3 Comparative Protocol Analysis: A2A and ACP

To highlight the limitations of MCP, it is useful to contrast it with protocols explicitly designed to facilitate deeper semantic interaction between autonomous agents.

Google's A2A (Agent-to-Agent): In stark contrast to MCP's centralized model, A2A is architected for peer-to-peer, asynchronous communication.[6] It is built to support multi-agent ecosystems where autonomous agents can engage in direct, trust-based collaboration. Key features include dynamic capability discovery, allowing agents to find and interact with each other at runtime, and support for long-lived, stateful interactions. This architecture is

designed for scenarios involving delegation and negotiation, such as a project management agent delegating subtasks to specialized coding and QA agents.[7]

IBM's ACP (Agent Communication Protocol): ACP represents a hybrid approach, combining a centrally managed client-server architecture with a strong focus on semantics.[7] Its primary goal is to enable deep semantic intent exchange through structured, multi-turn dialogues. ACP achieves this by leveraging shared ontologies and maintaining contextual awareness throughout an interaction. This makes it particularly well-suited for complex, cross-departmental enterprise workflows, such as HR onboarding, where agents representing different business units must coordinate through a shared, unambiguous understanding of the process.[7]

The following table provides a consolidated comparison of these three influential protocols, illustrating the critical trade-offs in their design philosophies.

Feature	**MCP (Anthropic)**	**A2A (Google)**	**ACP (IBM)**
Architecture	Centralized (Client-Server)	Peer-to-Peer (Asynchronous)	Client-Server with Central Management
Primary Focus	Agent-to-Tool Integration (Syntactic)	Agent-to-Agent Trust & Delegation (Collaborative)	Semantic Multi-Agent Dialogue (Semantic)
State Management	Intentionally Stateless	Stateful (Supports long-lived interactions)	Session-Aware (Contextual awareness)
Discovery Mechanism	Static (Predefined Metadata)	Dynamic (Capability Discovery)	Session-Aware Workflows
Semantic Support	Low (Typed schemas, assumes known context)	Medium (Enables negotiation, semantics are emergent)	High (Relies on shared ontologies for explicit meaning)
Key Trade-off	Simplicity & Scalability vs.	Autonomy & Flexibility vs.	Semantic Richness vs. Overhead &

	Flexibility & Autonomy	Coordination Complexity	Ontology Dependency

The choice of a communication protocol is not a neutral technical decision; it fundamentally encodes a philosophy of agent collaboration. MCP's design reflects a worldview centered on a single, powerful orchestrator commanding a suite of simple, stateless "tools." This is a master-servant model of interaction. In contrast, protocols like A2A and ACP are designed for a "society of equals," where autonomous peers negotiate and collaborate to achieve shared goals. The fragmentation in the current agentic landscape stems directly from the community's failure to reconcile these two profoundly different conceptions of what an "agent" is and how it should interact with its peers.

This divergence in protocol design is, in turn, a reflection of the underlying architecture of the LLMs themselves. Because today's leading models (like Gemini and Claude) are architected as powerful, centralized planners, it is natural for developers to design protocols that treat all external entities as simple tools to be invoked. The prevalence of MCP-like, tool-centric protocols is a direct consequence of this centralized cognitive architecture. A widespread shift towards true peer-to-peer semantic negotiation will likely only occur in tandem with a move towards more decentralized AI models, where intelligence and agency are not concentrated in a single monolithic entity. The protocol landscape is a mirror, and it currently reflects a deep centralization in our models of intelligence.

Section 3: Context as the Cognitive Battlefield: Propagation, Engineering, and Shared Realities

3.1 The Discipline of Context Engineering

If protocols are the pipes of multi-agent communication, then context is the fluid that flows through them. The success or failure of any collaborative task hinges on the quality, relevance, and shared interpretation of this context. This has given rise to a new and critical discipline: **Context Engineering**. It moves beyond the static nature of "prompt engineering" to encompass the dynamic, automated management of information flow to, from, and between agents in a complex system.[22] It is the practical, hands-on discipline that seeks to solve the theoretical problem of semantic alignment.

At its core, context engineering is guided by two fundamental principles. First is the imperative to **share full agent traces, not just individual messages**.[22] A final message from an agent is merely the conclusion of a reasoning process. The true context is embedded in the entire history of that process, including previous conversational turns, tool calls made, and errors encountered. Sharing only the final output strips away this vital nuance and invites misinterpretation by downstream agents. Second is the recognition that

actions carry implicit decisions.[22] When an agent chooses to call a specific API or format a response in a certain way, it is making a decision based on its understanding of the context. If multiple agents make such decisions in isolation due to fragmented context, their actions will inevitably conflict, leading to system-level failures.[22]

A primary driver for sophisticated context management is the physical limitation of the context window. Even with million-token capacities, long-running agentic processes can quickly generate enough conversational history, tool outputs, and state information to risk context overflow.[22] This forces developers to design intelligent strategies for context compression, storage, and retrieval, making context engineering the number one job of those building reliable AI agents.[22]

3.2 Shared Mental Models (SMMs): The Gold Standard for Alignment

To understand the ultimate goal of context engineering, it is essential to draw upon decades of research from the field of human-agent teaming and cognitive science. The theoretical gold standard for effective collaboration is the establishment of a **Shared Mental Model (SMM)**.[13] An SMM is not a vague notion of "being on the same page," but a formal construct with a precise definition and measurable components.

Definition: An SMM is defined as a set of knowledge structures held by members of a team that enable them to form accurate explanations for past events and accurate expectations for future events, which in turn allows them to coordinate their actions and adapt their behavior to the demands of the task and their teammates.[13] Crucially, SMM theory does not require that team members hold

identical mental models. Rather, it requires that their models are *compatible* enough to lead to common expectations.[13]

Components: SMMs are typically decomposed into several key sub-models that cover both the task and the team dynamics [14]:

- **Task Model:** A shared understanding of the task itself, including procedures, likely

contingencies, and strategies.
- **Equipment Model:** Shared knowledge about the tools being used, their functions, and their limitations.
- **Team Interaction Model:** A shared understanding of roles, responsibilities, communication patterns, and information flow within the team.
- **Team Member Model:** Knowledge about the specific capabilities, skills, knowledge, and even preferences of other team members.

Importance: The establishment of SMMs is paramount because it is directly and positively correlated with team performance.[13] When agents possess a strong SMM, they can predict the needs and actions of their teammates with high accuracy. This facilitates more efficient collaboration by enabling them to anticipate information needs, perform dynamic task allocation with minimal negotiation, and proactively share relevant information without being explicitly asked.[25] An SMM is the cognitive foundation upon which effective, fluid teamwork is built.

3.3 Architectural Patterns and Their Impact on Context Management

The choice of a multi-agent system architecture has a profound impact on the system's ability to form, maintain, and propagate the context needed for a coherent SMM. Different architectural patterns create different trade-offs between control, autonomy, and the cognitive overhead required to maintain alignment.

Hierarchical Systems: In a hierarchical architecture, a central "manager" or "orchestrator" agent decomposes a high-level goal, delegates subtasks to specialized "worker" agents, and integrates their results.[27] This pattern provides a centralized point for context management. The manager agent holds the primary, global mental model of the task and propagates curated, task-specific slices of context down to the workers.[30] This top-down control can be highly effective at enforcing a consistent worldview and preventing semantic drift among the workers. However, this centralization creates a single point of failure and a potential communication bottleneck. If the manager agent becomes overloaded or develops a flawed understanding of the task, that error will cascade throughout the entire system.[27]

Peer-to-Peer (Distributed/Decentralized) Systems: In a peer-to-peer architecture, agents operate autonomously without a central leader, communicating directly with one another as needed.[27] This model is more resilient, adaptable, and avoids the single point of failure inherent in hierarchical systems. However, the challenge of maintaining a consistent shared context—and thus a coherent SMM—is significantly greater. Context is fragmented across the network, and its propagation relies on emergent, ad-hoc interactions. Without robust and well-defined communication protocols, these systems are highly susceptible to confusion,

conflicting actions, and semantic drift, as individual agents may evolve their local mental models in divergent directions.[27]

Federated Systems: Federated architectures offer a compelling hybrid model that seeks to balance the trade-offs between centralized and decentralized approaches. In this pattern, individual agents (or groups of agents) maintain their own local data, models, and context to preserve privacy and autonomy. However, they coordinate through a central server that facilitates model aggregation or knowledge sharing without accessing the raw local data.[32] This model is particularly promising for context propagation and semantic alignment. A leading-edge example involves a federated approach where a central access point shares a common

semantic encoder, providing a baseline for meaning. Individual user devices then train local *semantic equalizers* to fine-tune their internal representations to align with this shared encoder.[10] This allows the system to foster mutual understanding and align the agents' internal latent spaces—a deep form of SMM—in a decentralized and privacy-preserving manner.[33]

A fundamental tension exists between the theoretical ideal of a perfectly shared mental model and the practical realities of distributed computing. This can be conceptualized as an "SMM-architecture impedance mismatch." A truly identical SMM, shared in its entirety across all agents at all times, would imply a centralized, monolithic state, thereby defeating the very purpose of employing a multi-agent system. The core principles of distributed systems are built on local views and decentralized control, where full state replication is prohibitively expensive and complex.[34]

Hierarchical systems attempt to resolve this mismatch by enforcing a compatible SMM top-down, but this comes at the cost of worker autonomy and creates performance bottlenecks. Peer-to-peer systems allow for compatibility to emerge from the bottom up, but this introduces the significant risk of divergence and cognitive fragmentation. Therefore, the central architectural challenge is not to achieve a perfect SMM, but a *sufficiently compatible* one.[13] This reframes the problem: the goal is to design communication and synchronization protocols that enable agents to propagate

just enough context to ensure their mental models remain aligned for the task at hand, without incurring the immense overhead of full state synchronization. The engineering focus must shift from the mechanism of "context propagation" to the goal of "Shared Mental Model maintenance." This requires developing formal frameworks for quantifying the semantic similarity between agents' mental models, which would allow a system to dynamically negotiate the level of context propagation required based on a continuous measurement of SMM divergence.

Section 4: The Large Context Window Paradox: Gemini CLI and the Mirage of Shared Understanding

4.1 The Promise: A Universal Context Canvas

The advent of Large Language Models with massive context windows, exemplified by Gemini 1.5 Pro's capacity for 1 to 2 million tokens, represents a paradigm shift in AI capabilities.[35] This enormous "cognitive canvas" allows a single agent to ingest and reason across entire codebases, extensive project histories, or libraries of documentation in a single pass.[15] This capability for powerful in-context learning, where the model learns from the provided information without requiring fine-tuning, unlocks use cases previously thought impossible.[36] For multi-agent systems, this development appears to offer a simple and elegant solution to the problem of context management: a universal blackboard or shared memory space where all relevant information for all agents can coexist, seemingly eliminating the need for complex propagation mechanisms.[37]

4.2 The Peril: Information vs. Understanding

This apparent solution, however, masks a deeper problem, giving rise to the **Large Context Window Paradox**: providing a shared *information space* does not guarantee a shared *understanding* or a shared *mental model*. The assumption that more context is always better is a fallacy rooted in a misunderstanding of the cognitive limitations of current transformer architectures. Several well-documented phenomena reveal the risks of this approach.

The "Lost in the Middle" Problem: A significant body of research has demonstrated that LLMs exhibit a strong positional bias. They pay disproportionately high attention to information presented at the very beginning and very end of a long context prompt, while information "lost in the middle" is frequently ignored, poorly recalled, or misrepresented.[8] This attentional decay means that two agents operating on the exact same large context may derive completely different understandings simply by focusing on different parts of the prompt. One agent might anchor its reasoning on the initial instructions, while another might be more influenced by the most recent data appended at the end, leading to divergent and misaligned outputs.

Context Distraction and Noise: The sheer volume of information in a million-token window can degrade, rather than enhance, model performance. An excessively large or irrelevant context introduces noise that can confuse the model and overwhelm its training.[24] As the signal-to-noise ratio decreases, it becomes computationally harder for the model to identify the most salient and relevant pieces of information for the task at hand.[41] Effective context management, therefore, requires not just providing information but actively filtering out noise.

Context Clash and Poisoning: A large, shared context is susceptible to internal contradictions. When different source files, historical data, or the outputs from previous agent turns contain conflicting or subtly inconsistent information, it creates a "context clash" that can make the model's behavior erratic and unpredictable.[24] More insidiously, if a hallucination or a factual error generated by one agent is preserved within the long-term context, it can "poison" all subsequent reasoning by other agents that rely on that context, leading to cascading failures that are extremely difficult to trace.[24]

4.3 Multi-Agent Systems in a Large-Context World

These challenges raise a critical architectural question: does the availability of a massive context window render the multi-agent paradigm obsolete? The community is currently engaged in a vigorous debate on this point, with two main schools of thought emerging.

The Single-Agent Argument: Proponents of this view, notably from Cognition AI, argue that a single agent operating within a massive, continuous context is inherently more reliable than a team of collaborating agents.[22] Their rationale is that multi-agent systems introduce failure points at every communication step and suffer from "dispersed decision-making," where a lack of thoroughly shared context leads to conflicting actions and fragile outcomes.[22] From this perspective, the ideal architecture is a single, coherent cognitive thread that avoids the overhead and unreliability of inter-agent communication altogether.

The Multi-Agent Argument: In contrast, proponents from institutions like Anthropic and Google argue that multi-agent systems are a vital and necessary strategy for scaling *token usage* and parallelizing reasoning for complex tasks that exceed the capacity of even the largest single context window.[43] Research from Anthropic has shown that distributing work across multiple agents, each with its own separate and more focused context window, can significantly outperform a single-agent approach, primarily because it allows for a greater total volume of tokens to be processed in parallel.[24] Furthermore, frameworks like Google's Chain-of-Agents (CoA) are explicitly designed to decompose long documents for sequential processing by a chain of worker agents. This approach has been shown to outperform a single large-context model by mitigating the "lost in the middle" problem; each agent in the

chain operates on a smaller, more manageable chunk of text, ensuring that all information is processed with high fidelity before being aggregated by a final manager agent.[40]

The debate between a single large-context agent and a multi-agent system presents a false dichotomy. Both architectural patterns are fundamentally attempting to manage the same constraint: the model's finite *attentional and reasoning capacity*. A large context window is analogous to a computer having a vast amount of RAM, but it does not increase the processing power of the CPU. It can hold more information, but it cannot necessarily reason about all of it effectively at once. Multi-agent systems, in this analogy, are a form of parallel processing, breaking a large problem into smaller pieces that can be handled by individual cores.

The optimal solution is therefore not one or the other, but a hybrid architecture that intelligently combines both. Such a system would use a "manager" agent operating with the full, large context to perform high-level strategic planning and task decomposition. This manager would then dynamically construct and populate the smaller, more focused context windows of "worker" agents with only the most relevant *slices* of the global context required for their specific sub-tasks. This approach leverages the global awareness afforded by a large context window while capitalizing on the focused reasoning of specialized agents. It treats the context window not as a passive data buffer to be stuffed with information, but as an active cognitive workspace to be managed and structured.

This perspective reveals that the very concept of a flat, monolithic "context window" is an architectural limitation inherited from the single-agent paradigm. It is a brute-force solution to the memory problem. The future of agentic architecture will likely move towards more sophisticated, structured cognitive architectures. Systems like the Semantic Hierarchical Memory Index (SHIMI) point the way forward, proposing a dynamic tree of semantic concepts where agents retrieve information based on meaning and abstraction level, rather than by its linear position in a text buffer.[46] The large context window is a powerful but transitional technology that will ultimately be superseded by these more advanced models of memory and knowledge representation.

Section 5: Cognitive Debt: Quantifying the Human Cost of Agent Misalignment

5.1 Defining Cognitive Debt in AI Systems

The persistent semantic misalignments within and between AI agents impose a significant, often hidden, cost. This cost is not measured in computational resources or API calls, but in the finite cognitive capacity of the human developers tasked with building, maintaining, and trusting these systems. This report formally defines this phenomenon as **Cognitive Debt**: the implicit, long-term cost of intellectual effort required by developers to bridge the gap between a multi-agent system's designed intent and its actual, often unpredictable, emergent behavior.

This concept is analogous to technical debt in traditional software engineering. However, instead of accumulating from poorly written code that will require future refactoring, cognitive debt accumulates from unresolved semantic ambiguities and context conflicts within the AI system. This debt is "paid down" not by rewriting code, but by developers expending immense mental energy to debug opaque interactions, reconcile conflicting agent states, and reverse-engineer the collective reasoning of a system that lacks a coherent, shared mental model. Each instance of misalignment adds to this debt, increasing developer frustration, slowing down progress, and fundamentally eroding trust in the AI system's reliability.[1]

5.2 The Developer Experience: Cognitive Dissonance and Load

The accumulation of cognitive debt has a direct and measurable psychological impact on the developers who interact with these systems. Two key concepts from cognitive science—cognitive dissonance and cognitive load—provide a powerful lens for understanding this impact.

Cognitive Dissonance: This psychological theory describes the mental stress and discomfort experienced by an individual who holds two or more contradictory beliefs, ideas, or values simultaneously.[47] In the context of multi-agent systems, developers experience cognitive dissonance when the system's documented, "believed" behavior (e.g., "the agents will collaborate to optimize the supply chain") is in direct conflict with its actual, observed behavior (e.g., "the logistics agent and the inventory agent are locked in a bidding war, driving up costs"). To resolve this dissonance, the developer is forced into a costly mental process: they must either change their belief about the system's design, concluding it is fundamentally flawed, or rationalize the aberrant behavior as a one-off bug, potentially ignoring a deeper systemic issue.[47] This continuous cycle of expectation-versus-reality mismatch is a core component of cognitive debt.

Cognitive Load Interference: Recent research demonstrates that forcing a single cognitive agent—be it human or AI—to interleave high-level, abstract reasoning with low-level, precise, detail-oriented tasks significantly degrades performance on the high-level reasoning task.[49]

This phenomenon, termed "cognitive load interference," explains why debugging multi-agent systems is uniquely challenging. The developer must simultaneously operate at multiple levels of abstraction: reasoning about the high-level strategic goals and interaction patterns of the entire agent team, while also tracing low-level data transformations, API call parameters, and subtle variations in prompt formatting. This constant context-switching between the strategic and the tactical imposes an immense cognitive load, making it difficult to form a clear mental model of the system's failure state.

A secondary form of cognitive debt also emerges from the developer's relationship with the AI itself. An over-reliance on AI agents for routine problem-solving and code generation can lead to the atrophy of the developer's own critical thinking and debugging skills.[3] This creates a dangerous paradox: the very systems that are increasing the complexity of the debugging task are simultaneously eroding the fundamental skills required to perform it. When these complex, misaligned systems inevitably fail, the developer may find themselves without the cognitive tools to fix them.[15]

5.3 The Practical Challenge: Debugging Emergent Chaos

The theoretical cognitive burdens on developers manifest as concrete, formidable challenges in the day-to-day practice of debugging multi-agent systems. The nature of these systems makes them resistant to traditional debugging methodologies.

Complex Interactions & Cascading Errors: In a monolithic system, the scope of a bug is often localized. In a multi-agent system, errors are frequently non-local and emergent. An incorrect piece of context generated by an upstream "researcher" agent can cause a downstream "writer" agent to produce a factually incorrect report in a way that is seemingly unrelated to the initial error.[51] The failure cascades through the system, making root-cause analysis a forensic exercise that can span multiple agents, tools, and conversational turns.[53]

Unpredictable & Non-Deterministic Outputs: The inherent stochasticity of LLMs is a known challenge, but it is amplified exponentially in a multi-agent setting. A slight, random variation in the output of one agent can send the entire system down a completely different execution path. The same initial input can produce wildly different system trajectories and final outcomes across multiple runs.[51] This non-determinism makes bugs incredibly difficult to reproduce, rendering traditional debugging tools like breakpoints and deterministic unit tests largely ineffective.[43]

Opaque Reasoning Paths: The most significant challenge is the opacity of the agents' decision-making processes. Without specialized tooling, the "why" behind an agent's choice to use a particular tool or generate a specific response is often hidden within its latent

space.[52] Debugging these systems requires a new class of tools focused on

agent tracing. These tools must move beyond code execution and provide visibility into the cognitive processes of the system: the flow of context, the history of inter-agent messages, and the reasoning paths that led to a particular decision. Interactive debuggers like AGDebugger, which allow developers to visualize message histories, reset workflows to earlier states, and edit agent messages to test hypotheses, represent the future of debugging in the agentic era.[52]

The concept of cognitive debt can be operationalized as a key performance indicator (KPI) for evaluating the health and maintainability of a multi-agent architecture. It can be measured through proxy metrics such as the mean-time-to-resolution for agent-related bugs, the number of distinct agent traces a developer must inspect to diagnose a single failure, or the frequency of "unexplained" emergent behaviors that require manual intervention. A system that is transparent and easy to debug can be said to have low cognitive debt, while a system that is opaque and unpredictable carries a high cognitive debt liability.

From this perspective, architectural decisions should be evaluated not only on their ability to produce the correct answer but also on how difficult they make it to understand why a wrong answer was produced. This reframes the Human-in-the-Loop (HITL) paradigm. Often presented as a safety feature or a solution for agent failures, frequent HITL interventions can also be viewed as a *symptom* of high cognitive debt.[55] When a system constantly needs to pause and ask for human approval or clarification, it is not truly collaborating; it is offloading its internal inconsistencies and semantic uncertainties onto the human user's cognitive balance sheet. A well-aligned, low-debt system should operate with minimal need for human intervention. Therefore, the frequency and nature of HITL triggers could serve as a powerful, quantifiable metric for a system's accumulated cognitive debt.

Section 6: Architecting for Alignment: Frameworks for Semantic Cohesion

6.1 Data-Level Solutions: Grounding Agents in a Shared Reality

The foundation of semantic alignment is a shared, unambiguous understanding of the data upon which agents operate. Without this grounding, agents are liable to hallucinate, misinterpret, and diverge. Data-level solutions aim to create a "single source of truth" that

serves as a common reference point for all agents in a system.

Semantic Layers: A universal semantic layer acts as a bridge between raw, technical data sources and the world of business concepts.[9] It provides a unified, virtualized interface that maps complex data schemas to a consistent, governed vocabulary of business definitions, metrics, and contexts. For AI agents, this is critical. Instead of interacting with ambiguous or fragmented data, they query a layer that provides clear, standardized terms and relationships. This grounding in shared business logic dramatically reduces errors. For example, research from AtScale has shown that employing a semantic layer can triple the accuracy of AI-generated SQL queries compared to querying raw databases directly, as it prevents the model from making incorrect assumptions about table joins or column meanings.[9] By embedding business context directly into the analytics pipeline, a semantic layer ensures that AI outputs are not only accurate but also trustworthy and explainable.[9]

Verified Semantic Caches: This technique provides a pragmatic, first-line defense against hallucinations for known queries. The system maintains a curated knowledge base of verified question-and-answer pairs.[57] When a user query is received, it is first compared against this cache using a semantic similarity search. If the query has a strong semantic match (e.g., >80% similarity) with a question in the cache, the system returns the pre-verified, trusted answer directly, bypassing the LLM entirely. If the match is partial (e.g., 60-80% similarity), the cached answer can be injected into the LLM's context as a strong piece of evidence to ground its response. This approach significantly improves response accuracy for common queries, reduces latency and costs, and acts as a crucial guardrail against generating incorrect information for well-understood topics.[57]

6.2 Model-Level Solutions: Aligning Internal Representations

While grounding data is essential, deeper alignment requires addressing the internal representations within the models themselves. These solutions aim to ensure that different agents develop compatible internal "worldviews."

Federated Semantic Alignment: This novel approach, particularly relevant for decentralized systems where data privacy is a concern, seeks to align the agents' internal latent spaces without sharing the underlying private data.[10] In this federated model, a central access point distributes a shared

semantic encoder, which provides a common baseline for interpreting meaning. Each user device or agent then trains a lightweight, local *semantic equalizer*. This equalizer acts as an adapter, fine-tuning the agent's local internal representations to be more compatible with the shared encoder. This process fosters mutual comprehension and task-oriented

communication by aligning the agents' semantic "languages" at a deep, mathematical level, all while adhering to power, complexity, and privacy constraints.[10]

Multitask Training: A key challenge in multi-agent communication is "semantic drift," the gradual process by which the meaning of messages in an agent's latent language deviates over time, leading to miscoordination.[58] Research has shown that multitask training is an effective countermeasure. By training a population of agents on a diverse set of shared goals and communicative needs, rather than a single, narrow task, the semantics of their communication protocols are forced to become more stable and generalizable. This diversity of tasks acts as a stabilizing pressure, inhibiting semantic drift and, in some cases, improving overall sample efficiency and model performance.[58]

6.3 Protocol-Level Solutions: Explicit Meaning and Negotiation

The communication protocols themselves can be designed to enforce semantic clarity and enable dynamic alignment. These solutions move beyond the static, syntactic contracts of protocols like MCP.

Shared Ontologies and Ontology Alignment: Formal ontologies provide a powerful mechanism for reducing ambiguity in communication. By creating a precise, machine-readable definition of concepts, their properties, and the relationships between them, a shared ontology can serve as a formal contract for meaning within a multi-agent system.[11] This ensures that when one agent sends a message containing the term "customer," all other agents interpret it according to the same, unambiguous definition.[62] Furthermore, LLMs themselves can be leveraged to facilitate the alignment of different ontologies. A multi-agent dialogue model can be employed where agents, each with their own domain-specific ontology, negotiate correspondences with the assistance of an LLM, making the process more interpretable and reducing the need for costly human expert involvement.[63]

Dynamic Context Negotiation: In contrast to systems where context is passively pushed to agents, more advanced protocols allow agents to actively negotiate the context they require to perform a task. This involves communication patterns like proposing, accepting, rejecting, and counter-proposing, enabling agents to dynamically adapt their information-sharing strategy based on task complexity, resource constraints, and their current understanding.[64] For instance, a framework for Adaptive Model Context Protocols (AMCP) uses a hierarchical context model and bidirectional flow, allowing agents to request specific types of context at varying levels of abstraction, intelligently filtering information exchange to avoid overload while ensuring task performance.[67]

6.4 System-Level Solutions: Enforcing Consistency Through Transactional Guarantees

Even with aligned data, models, and protocols, complex, long-running workflows are vulnerable to partial failures that can leave the entire system in an inconsistent state. System-level solutions aim to provide robust guarantees of consistency and recoverability.

The SagaLLM Framework: A groundbreaking approach in this area is SagaLLM, a structured multi-agent architecture that integrates the Saga transactional pattern, a concept from the world of distributed databases, into the LLM planning process.[12] The Saga pattern addresses the challenge of long-lived transactions by decomposing a large workflow into a sequence of smaller, independently executed sub-transactions. Each sub-transaction has a corresponding

compensating transaction that can be executed to semantically undo its effects if a subsequent step fails.[69]

SagaLLM applies this pattern to multi-agent systems to address their foundational limitations: unreliable self-validation, context loss, and the lack of rollback capabilities.[12] The framework uses specialized validation agents, persistent memory to track state, and LLM-generated compensation logic. If any agent in a multi-step workflow fails (e.g., a "hotel booking" agent fails after a "flight booking" agent has already succeeded), the system can automatically trigger the compensating transaction (e.g., "cancel flight") to restore the entire system to a globally consistent state.[71] While this relaxes the strict ACID guarantees of traditional databases, it provides the essential workflow-wide consistency and recovery needed for reliable real-world applications.[69]

Achieving robust semantic alignment is not a matter of finding a single "silver bullet" solution. Instead, it requires a comprehensive, defense-in-depth strategy that operates across all layers of the system stack. The data must be grounded in a shared reality through semantic layers. The models' internal representations must be aligned using techniques like federated learning. The communication protocols must be explicit and unambiguous, leveraging formal ontologies. Finally, the overall system must possess safeguards, such as transactional frameworks, to enforce consistency even in the face of failure.

This multi-layered approach represents a significant trend: the re-introduction of rigorous principles from classical computer science and distributed systems—such as strong schemas, transactional integrity, and formal semantics—into the more fluid, probabilistic world of LLMs. The evolution of agentic AI is not a complete departure from the past, but rather a necessary synthesis. The future of reliable AI engineering will be defined by our ability to find the optimal balance between the generative, flexible intelligence of LLMs and the formal, verifiable

guarantees of traditional computer science. Frameworks like SagaLLM are a powerful embodiment of this critical synthesis.

Section 7: Conclusion and Strategic Recommendations

7.1 Synthesis: From Fragmented Landscape to Cohesive Ecosystem

This report has systematically deconstructed the challenge of semantic misalignment in multi-agent AI systems. The analysis confirms that while technological progress—from million-token context windows to standardized protocols like MCP—has successfully addressed syntactic interoperability, it has simultaneously exposed and exacerbated a deeper semantic divide. The transition from single, powerful agents to interconnected multi-agent systems has elevated semantic alignment from a secondary concern to the central challenge for building reliable, scalable, and trustworthy AI.

The concept of **cognitive debt** has been formalized as the primary hidden cost of failing to address this challenge. It represents the cumulative cognitive load placed upon human developers who must manually bridge the semantic gaps left by their AI systems. This debt manifests in protracted debugging cycles, a loss of trust in non-deterministic outputs, and an overall friction that impedes the successful deployment of agentic AI in mission-critical applications. The current agentic landscape is fragmented not because the pipes between agents are broken, but because there is no shared language or understanding flowing through them.

7.2 Recommendations for Platform Architects (e.g., Google, Anthropic)

The architects of foundational models and core AI platforms are uniquely positioned to address semantic misalignment at its source.

- **Elevate SMMs to a First-Class Construct:** The concept of a Shared Mental Model (SMM) must be elevated from a theoretical ideal to a concrete, first-class architectural

construct. This requires investing in research and developing tools that allow developers to explicitly define, monitor, and measure the alignment of SMMs between agents. Future platforms should move beyond simple context passing and provide mechanisms to quantify the semantic divergence between agents' internal states, enabling systems to self-correct when their worldviews drift apart.

- **Develop Hybrid Context Architectures:** The current monolithic, flat context window is a brute-force approach to memory. The next generation of platforms should support structured, hierarchical memory architectures.[46] This would enable more sophisticated, meaning-based context retrieval, allowing an agent to query for "the project's security constraints" rather than linearly scanning a million-token buffer. This shift from a data-centric to a concept-centric memory model is essential for managing cognitive load at scale.
- **Invest in Semantically Rich Protocols:** While MCP is valuable for tool integration, platform leaders must champion and contribute to open standards that are purpose-built for deep collaboration. Supporting and extending protocols that natively handle statefulness, dynamic context negotiation, and ontology-based communication (such as A2A and ACP) is critical for fostering a truly interoperable and semantically coherent ecosystem.[7]

7.3 Recommendations for Framework Developers (e.g., LangChain, Microsoft)

The developers of agent orchestration frameworks like LangGraph and AutoGen are on the front lines of managing cognitive debt.

- **Build for Debuggability:** The developer experience must be a primary design consideration. Frameworks should prioritize the development of advanced tracing, visualization, and interactive debugging tools that make the flow of context and reasoning between agents transparent and inspectable.[52] "Low cognitive debt" should be marketed and measured as a core feature, as it is a direct indicator of a framework's usability in complex, real-world scenarios.
- **Integrate Transactional Patterns:** The principles of transactional consistency and graceful failure recovery are not optional extras; they are essential for reliability. Frameworks should provide native, easy-to-use support for patterns like Sagas within their orchestration logic.[12] Giving developers the building blocks to create resilient, self-correcting workflows will dramatically reduce the cognitive burden of manual error handling and state reconciliation.
- **Offer Architectural Flexibility and Guidance:** There is no one-size-fits-all architecture for multi-agent systems. Frameworks should provide clear blueprints, components, and

documentation for a variety of patterns (hierarchical, peer-to-peer, federated) and, crucially, offer explicit guidance on the context management trade-offs, potential for semantic drift, and cognitive debt implications associated with each choice.

7.4 Recommendations for Enterprise Application Builders

The ultimate responsibility for building reliable systems lies with the teams deploying them in production.

- **Adopt a Defense-in-Depth Strategy:** Do not rely on a single solution to ensure alignment. A robust strategy must be multi-layered. Implement semantic layers to govern data and establish a single source of truth. Use explicit and detailed prompting to define agent roles and seed their initial SMMs. Choose an architectural pattern that matches the required collaboration style of the task. And, for all critical or irreversible actions, incorporate human-in-the-loop validation as a final safeguard.
- **Start with the Human Factor:** Design systems with the developer's and maintainer's cognitive load in mind. In the trade-off between full autonomy and system transparency, err on the side of transparency. Prioritize architectures that are easier to understand and debug, even if they seem theoretically less sophisticated. The most effective and sustainable AI system is one that its human counterparts can understand, trust, and confidently manage.
- **Quantify Cognitive Debt:** Begin to treat cognitive debt as a tangible metric. Track proxies such as the mean time to resolution for agent-related bugs, the number of conversational turns required for a successful outcome, and the frequency of required human intervention. Use this data to inform architectural decisions, justify investments in better tooling and frameworks, and make the hidden cost of semantic misalignment visible to all stakeholders.

Works cited

1. From fragmented to agentic: evolving your enterprise AI stack ..., accessed on September 2, 2025, https://hypermode.com/blog/enterprise-ai-strategy
2. Agentic AI Isn't Plug and Play: 5 Barriers to Success - Reworked, accessed on September 2, 2025, https://www.reworked.co/digital-workplace/why-agentic-ai-projects-fail/
3. AI in Education: Rethinking MIT's Cognitive Debt Claim - Coacharya, accessed on September 2, 2025, https://coacharya.com/uncategorized/ai-education-cognitive-debt/
4. Your Brain Is at Risk of Cognitive Debt Amid AI - Psychology Today, accessed on September 2, 2025, https://www.psychologytoday.com/us/blog/harnessing-hybrid-intelligence/20250

6/your-brain-is-at-risk-of-cognitive-debt-amid-ai
5. Cooperation as Black Box: Conceptual Fluctuation and Diagnostic ..., accessed on September 2, 2025, https://arxiv.org/pdf/2506.22876
6. How Agents Talk: Mapping the Future of Multi-Agent Communication ..., accessed on September 2, 2025, https://medium.com/software-architecture-in-the-age-of-ai/how-agents-talk-mapping-the-future-of-multi-agent-communication-protocols-6115ea083dba
7. AI Agent Ecosystem: A Guide to MCP, A2A, and Agent Communication Protocols - Addepto, accessed on September 2, 2025, https://addepto.com/blog/ai-agent-ecosystem-a-guide-to-mcp-a2a-and-agent-communication-protocols/
8. Why larger LLM context windows are all the rage - IBM Research, accessed on September 2, 2025, https://research.ibm.com/blog/larger-context-window
9. Semantic Layers: The Missing Link Between AI and Business Insight ..., accessed on September 2, 2025, https://medium.com/@axel.schwanke/semantic-layers-the-missing-link-between-ai-and-business-insight-3c733f119be6
10. SPAICOM/multi-agent-semantic-alignment: This repository tackles latent space misalignment in multi-agent AI-native semantic communications. It introduces a federated approach where an access point shares a semantic encoder, while user devices use local semantic equalizers to enhance mutual understanding. - GitHub, accessed on September 2, 2025, https://github.com/SPAICOM/multi-agent-semantic-alignment
11. Semantic Interoperability of Multi-Agent Systems in Autonomous ..., accessed on September 2, 2025, https://www.mdpi.com/2079-9292/14/13/2630
12. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2503.11951
13. (PDF) Shared Mental Models - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/226781970_Shared_Mental_Models
14. A framework for developing and using shared mental models in human-agent teams, accessed on September 2, 2025, https://hrilab.tufts.edu/publications/scheutzetal17smm.pdf
15. Gemini CLI: Coding with a Million-Token Context in Your IDE | by My Social - Medium, accessed on September 2, 2025, https://medium.com/aimonks/gemini-cli-coding-with-a-million-token-context-in-your-ide-0f483753d6f0
16. Why do AI agents communicate in human language? - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2506.02739v1
17. Multi-Agent AI Gone Wrong: How Coordination Failure Creates Hallucinations | Galileo, accessed on September 2, 2025, https://galileo.ai/blog/multi-agent-coordination-failure-mitigation
18. Misalignment-by-default in multi-agent systems - AI Alignment Forum, accessed on September 2, 2025, https://www.alignmentforum.org/posts/cemhavELfHFHRaA7Q/misalignment-by-d

efault-in-multi-agent-systems
19. Multi-Agent Communication Protocols: A Technical Deep Dive ..., accessed on September 2, 2025, https://geekyants.com/blog/multi-agent-communication-protocols-a-technical-deep-dive
20. Model Context Protocol (MCP) for Multi-Agent Systems | by Mahesh Nikam | Medium, accessed on September 2, 2025, https://medium.com/@nikammahesh01/model-context-protocol-mcp-for-multi-agent-systems-710575cf8db0
21. A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP) - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2505.02279v1
22. Don't Build Multi-Agents - Cognition, accessed on September 2, 2025, https://cognition.ai/blog/dont-build-multi-agents
23. Context Engineering in LLM-Based Agents | by Jin Tan Ruan, CSE Computer Science, accessed on September 2, 2025, https://medium.com/@jtanruan/context-engineering-in-llm-based-agents-d670d6b439bc
24. Context Engineering - LangChain Blog, accessed on September 2, 2025, https://blog.langchain.com/context-engineering-for-agents/
25. Implementing Shared Mental Models for Collaborative Teamwork - Intelligent Agents Laboratory, accessed on September 2, 2025, http://agentlab.psu.edu/lab/publications/cast_IAT03Wkshp.pdf
26. Full article: The role of shared mental models in human-AI teams: a theoretical review, accessed on September 2, 2025, https://www.tandfonline.com/doi/full/10.1080/1463922X.2022.2061080
27. Agentic Patterns: Architectures for Coordinated AI Systems | by The ..., accessed on September 2, 2025, https://medium.com/@learning_37638/agentic-patterns-architectures-for-coordinated-ai-systems-34d9d8d8e1e2
28. What are hierarchical multi-agent systems? - Milvus, accessed on September 2, 2025, https://milvus.io/ai-quick-reference/what-are-hierarchical-multiagent-systems
29. Hierarchical Multi-Agent Systems: Concepts and Operational Considerations - Over Coffee, accessed on September 2, 2025, https://overcoffee.medium.com/hierarchical-multi-agent-systems-concepts-and-operational-considerations-e06fff0bea8c
30. PosterForest: Hierarchical Multi-Agent Collaboration for Scientific Poster Generation - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2508.21720v1
31. Centralized vs Distributed Multi-Agent AI Coordination Strategies - Galileo AI, accessed on September 2, 2025, https://galileo.ai/blog/multi-agent-coordination-strategies
32. A Survey of Multi Agent Reinforcement Learning Federated Learning and

Cooperative and Noncooperative Decentralized Regimes - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2507.06278v1
33. Context-Based Concurrent Experience Sharing in Multiagent Systems - Manning College of Information & Computer Sciences, accessed on September 2, 2025, https://people.cs.umass.edu/~bsilva/papers/contextLearning_aamas2017_full.pdf
34. Multi-agent system - Wikipedia, accessed on September 2, 2025, https://en.wikipedia.org/wiki/Multi-agent_system
35. Gemini CLI Tutorial Series — Part 9: Understanding Context ..., accessed on September 2, 2025, https://medium.com/google-cloud/gemini-cli-tutorial-series-part-9-understanding-context-memory-and-conversational-branching-095feb3e5a43
36. Long context | Gemini API | Google AI for Developers, accessed on September 2, 2025, https://ai.google.dev/gemini-api/docs/long-context
37. Long context | Generative AI on Vertex AI - Google Cloud, accessed on September 2, 2025, https://cloud.google.com/vertex-ai/generative-ai/docs/long-context
38. Gemini 1.5 Pro just got 2x better: Get access to a two million token context window - Medium, accessed on September 2, 2025, https://medium.com/around-the-prompt/gemini-1-5-pro-just-got-2x-better-11ccc573031b
39. LLMs with largest context windows - Codingscape, accessed on September 2, 2025, https://codingscape.com/blog/llms-with-largest-context-windows
40. Chain of Agents: Large Language Models Collaborating on Long-Context Tasks - OpenReview, accessed on September 2, 2025, https://openreview.net/pdf?id=LuCLf4BJsr
41. What Causes Poor Long-Context Performance? : r/LocalLLaMA - Reddit, accessed on September 2, 2025, https://www.reddit.com/r/LocalLLaMA/comments/1lykf92/what_causes_poor_long context_performance/
42. Understanding the Impact of Increasing LLM Context Windows - Meibel, accessed on September 2, 2025, https://www.meibel.ai/post/understanding-the-impact-of-increasing-llm-context-windows
43. How we built our multi-agent research system - Anthropic, accessed on September 2, 2025, https://www.anthropic.com/engineering/built-multi-agent-research-system
44. Chain of Agents: Large language models collaborating on long-context tasks, accessed on September 2, 2025, https://research.google/blog/chain-of-agents-large-language-models-collaborating-on-long-context-tasks/
45. NeurIPS Poster Chain of Agents: Large Language Models Collaborating on Long-Context Tasks, accessed on September 2, 2025, https://neurips.cc/virtual/2024/poster/95563
46. SHIMI, a Semantic Hierarchical Memory Index for Scalable Agent ReasoningSupported by Concia - arXiv, accessed on September 2, 2025,

https://arxiv.org/html/2504.06135v1
47. Cognitive dissonance - Wikipedia, accessed on September 2, 2025, https://en.wikipedia.org/wiki/Cognitive_dissonance
48. Neighborhood Cognition Consistent Multi-Agent Reinforcement Learning - Association for the Advancement of Artificial Intelligence (AAAI), accessed on September 2, 2025, https://cdn.aaai.org/ojs/6212/6212-13-9437-1-10-20200516.pdf
49. arxiv.org, accessed on September 2, 2025, https://arxiv.org/html/2508.08882v1
50. The hidden cost of AI: Why "Cognitive Debt" is a warning for ..., accessed on September 2, 2025, https://medium.com/@RhoBus/the-hidden-cost-of-ai-why-cognitive-debt-is-a-warning-for-developers-489078b4dfcf
51. Real-Time Debugging for Multi-Agent LLM Pipelines | newline, accessed on September 2, 2025, https://www.newline.co/@zaoyang/real-time-debugging-for-multi-agent-llm-pipelines--bb853aed
52. Agent Tracing for Debugging Multi-Agent AI Systems - Maxim AI, accessed on September 2, 2025, https://www.getmaxim.ai/articles/agent-tracing-for-debugging-multi-agent-ai-systems/
53. Interactive Debugging and Steering of Multi-Agent AI Systems - Adam Fourney, accessed on September 2, 2025, https://www.adamfourney.com/papers/epperson_chi2025.pdf
54. Interactive Debugging and Steering of Multi-Agent AI Systems - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/391240139_Interactive_Debugging_and_Steering_of_Multi-Agent_AI_Systems
55. Implement human-in-the-loop confirmation with Amazon Bedrock Agents - AWS, accessed on September 2, 2025, https://aws.amazon.com/blogs/machine-learning/implement-human-in-the-loop-confirmation-with-amazon-bedrock-agents/
56. Human-in-the-Loop for AI Agents: Best Practices, Frameworks, Use Cases, and Demo, accessed on September 2, 2025, https://www.permit.io/blog/human-in-the-loop-for-ai-agents-best-practices-frameworks-use-cases-and-demo
57. Reducing hallucinations in LLM agents with a verified semantic ..., accessed on September 2, 2025, https://aws.amazon.com/blogs/machine-learning/reducing-hallucinations-in-llm-agents-with-a-verified-semantic-cache-using-amazon-bedrock-knowledge-bases/
58. Multitasking Inhibits Semantic Drift - DSpace@MIT, accessed on September 2, 2025, https://dspace.mit.edu/bitstream/handle/1721.1/142873/2021.naacl-main.421.pdf?sequence=2&isAllowed=y
59. Agent Systems: Managing Shift, Drift, and Tool Confusion | by Shashanka B R |

Medium, accessed on September 2, 2025, https://medium.com/@shashanka_b_r/evolution-and-alignment-in-multi-agent-systems-managing-shift-drift-and-tool-confusion-04c6ce42af5a
60. [2104.07219] Multitasking Inhibits Semantic Drift - arXiv, accessed on September 2, 2025, https://arxiv.org/abs/2104.07219
61. Ontology-based Open Multi-agent Systems for Adaptive Resource Management - SciTePress, accessed on September 2, 2025, https://www.scitepress.org/Papers/2020/88963/88963.pdf
62. Transfer Learning Method Using Ontology for Heterogeneous Multi-agent Reinforcement Learning - SciSpace, accessed on September 2, 2025, https://scispace.com/pdf/transfer-learning-method-using-ontology-for-heterogeneous-3muuihsel9.pdf
63. (PDF) Large Language Model Assisted Multi-Agent Dialogue for ..., accessed on September 2, 2025, https://www.researchgate.net/publication/378332970_Large_Language_Model_Assisted_Multi-Agent_Dialogue_for_Ontology_Alignment
64. (PDF) Multi-agent Systems Negotiation to deal with Dynamic scheduling in Disturbed industrial context - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/337679746_Multi-agent_Systems_Negotiation_to_deal_with_Dynamic_scheduling_in_Disturbed_industrial_context
65. Multi-Agent Systems and Negotiation: Strategies for Effective Agent Collaboration, accessed on September 2, 2025, https://smythos.com/developers/agent-development/multi-agent-systems-and-negotiation/
66. A Multi-Agent Based Framework for Supporting Learning in Adaptive Automated Negotiation. - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/220708682_A_Multi-Agent_Based_Framework_for_Supporting_Learning_in_Adaptive_Automated_Negotiation
67. (PDF) Adaptive Model Context Protocols for Multi-Agent Collaboration - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/392557837_Adaptive_Model_Context_Protocols_for_Multi-Agent_Collaboration
68. AgentFlow: A Context Aware Multi-Agent Framework for Dynamic Agent Collaboration - SciTePress, accessed on September 2, 2025, https://www.scitepress.org/Papers/2025/133757/133757.pdf
69. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning - arXiv, accessed on September 2, 2025, https://arxiv.org/pdf/2503.11951
70. [2503.11951] SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning - arXiv, accessed on September 2, 2025, https://arxiv.org/abs/2503.11951
71. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2503.11951v2
72. SagaLLM: Context Management, Validation, and Transaction Guarantees for

Multi-Agent LLM Planning - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2503.11951v1
73. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning | alphaXiv, accessed on September 2, 2025, https://www.alphaxiv.org/overview/2503.11951